

13

AI Infrastructure Engineering

High-Performance AI Infrastructure & Serving

IN THIS WEEK · 3 TOPICS · 39 SECTIONS

01

GPU Computing Fundamentals

13 SECTIONS

- 01

The Big Picture First
- 03

GPU Architecture
- 05

CUDA Fundamentals
- 07

GPU Memory
- 09

NVIDIA Driver Stack
- 11

cuDNN
- 13

GPU Scheduling
- 02

AI Infrastructure Overview
- 04

CPU vs GPU
- 06

CUDA Cores
- 08

Tensor Cores
- 10

CUDA Toolkit
- 12

NCCL

02

Production AI Serving

13 SECTIONS

- 01

The Big Picture First
- 03

TensorRT
- 05

Ollama
- 07

SGLang
- 09

Continuous Batching
- 11

Speculative Decoding
- 13

AI Serving Optimization
- 02

vLLM
- 04

TensorRT-LLM
- 06

Hugging Face TGI
- 08

LMDeploy
- 10

KV Cache
- 12

Model Quantization

03

Distributed AI Infrastructure

13 SECTIONS

- 01

The Big Picture First
- 03

Ray Serve
- 05

Distributed Inference
- 07

Kubernetes for AI
- 09

NVIDIA GPU Operator
- 11

Kubeflow
- 13

AI Infrastructure Monitoring
- 02

Ray Fundamentals
- 04

Ray Train
- 06

Distributed Training
- 08

GPU Scheduling in Kubernetes
- 10

KServe
- 12

Autoscaling AI Workloads

WEEK 13 OF 17

01

02

03

04

05

06

07

08

09

10

11

12

13

14

15

16

17

GPU Computing Fundamentals

The Big Picture First

Everything we've discussed throughout this entire series so far — RAG, agents, LLMOps, security — has assumed that somewhere, underneath all of it, there's actual physical computing hardware doing the real, heavy-lifting work of actually running these models. Week 12 finally opens up that hood and looks at what's actually happening down at the hardware level. Here's the core fact that explains basically everything in this whole explanation: training and running large language models involves doing an absolutely enormous number of relatively simple mathematical calculations (mostly multiplying and adding numbers together, over and over, at truly massive scale), and it turns out that regular computer processors, despite being genuinely powerful and flexible, are actually not the best-suited tool for this particular specific kind of work. A different kind of processor — the GPU — turns out to be dramatically better suited to it. Understanding why is really the foundation for understanding this entire week's material.

01 AI Infrastructure Overview

AI infrastructure refers to the complete stack of physical hardware and supporting software that actually makes it possible to train and run AI models at real, meaningful scale — everything from the actual physical computer chips doing the raw calculations, all the way up through the software layers that let a developer actually write and run their AI code without needing to personally understand every single detail of the underlying hardware.

It helps to think of this as a layered stack, where each layer builds on top of the one below it. At the very bottom sits the actual physical hardware — the specialized processors (which we'll get into shortly), the physical memory, and the networking that connects many machines together. Above that sits a layer of low-level software specifically designed to let higher-level programs actually communicate with and properly make use of that specialized hardware (we'll cover several of these pieces later in this explanation — the driver stack, the CUDA toolkit, and so on). And above that sits the higher-level software frameworks (like the popular ones such as PyTorch and TensorFlow, which you may have already come across if you've done any hands-on AI development) that most developers actually interact with directly, day to day, without needing to think too hard about most of what's happening underneath.

Why does this whole layered infrastructure matter so much? Because the specific hardware and software choices made at each one of these layers genuinely have an enormous, direct impact on how fast a model can actually be trained, how much it costs to actually run, and how many users it can genuinely serve at once. A team that deeply understands this whole stack can make genuinely much better decisions about what hardware to actually buy or rent, and how to actually configure it well, compared to a team that treats it all as a complete, mysterious black box they never really think about at all.

02 GPU Architecture

GPU stands for Graphics Processing Unit, and to understand why it's become so central to AI, it genuinely helps to understand where it actually originally came from. GPUs were originally built for an entirely different purpose — rendering graphics for video games and other visual applications, where a computer needs to

calculate the color and position of literally millions of individual pixels on a screen, many times every single second, to actually produce smooth, moving, visual images.

Here's the key architectural insight that turned out to matter enormously for AI, well beyond its original graphics purpose: calculating the value of one single pixel is a relatively simple, small, well-defined mathematical task, and, critically, the calculation for one pixel doesn't generally depend on the calculation for a different, other pixel — meaning millions of these simple calculations can genuinely all be done at the exact same time, completely independently and in parallel, rather than needing to be done strictly one after another. So GPU designers built chips specifically optimized around this exact insight — rather than having just a few, individually very powerful processing units (which is genuinely how a regular computer processor, a CPU, is actually built), a GPU instead contains many thousands of smaller, individually simpler processing units, all working together at the exact same time, in parallel.

It turns out that the core mathematical operations involved in running an AI model — things like multiplying huge grids of numbers together (called matrix multiplication, which is genuinely the fundamental core mathematical operation underlying almost everything a neural network actually does) — have this exact same basic character as rendering graphics pixels: an enormous number of relatively simple calculations that can genuinely all be done at the same time, in parallel, rather than one at a time, in sequence. This is precisely why GPUs, originally built for an entirely different purpose, turned out to be so remarkably, unexpectedly well-suited to AI workloads once people actually started trying to use them that way.

03 CPU vs GPU

Now that we understand the basic architectural difference, it's worth directly comparing CPUs and GPUs side by side, because understanding exactly when each one is genuinely the better tool for a given job is a genuinely foundational, important piece of understanding AI infrastructure well.

A CPU (Central Processing Unit) is the general-purpose "brain" of a regular computer, and it's specifically designed to be highly flexible and very good at handling a wide, genuinely diverse range of different, varied tasks, often one complex task quickly after another, in a fairly sequential order. A CPU typically has a relatively small number of individually very powerful, very sophisticated processing cores (commonly somewhere between around 4 and maybe 64 cores in a typical modern machine), each one capable of handling genuinely complex, varied logic and making rapid, sophisticated decisions.

A GPU, by contrast, as we just discussed, has thousands of individually much simpler processing cores, all specifically optimized for doing the exact same type of relatively simple calculation, over and over, on many different pieces of data all at the same time. A helpful, often-used analogy here: think of a CPU as being like a small team of a few genuinely brilliant, highly skilled individual specialists, each one capable of tackling a wide range of genuinely different, complex problems one after another. Think of a GPU as being like an enormous crowd of thousands of people, each one only capable of doing one simple task (like, say, basic addition), but who can all do that same simple task simultaneously, all at once, together.

For tasks that genuinely require complex, varied, sequential decision-making (like running a regular website's core application logic, or managing a computer's own overall operating system), a CPU is genuinely the better, more suitable tool. For tasks that genuinely involve doing the exact same relatively simple type of calculation an enormous number of times, in parallel (like training or running an AI model), a GPU is dramatically, genuinely faster — often by a factor of many multiples, or even many, many times faster, for exactly this specific kind of work.

Understanding that GPUs are well-suited to AI workloads is one thing — but a GPU's specialized hardware, all on its own, doesn't automatically know how to actually run AI code. CUDA (which stands for Compute Unified Device Architecture) is a platform and programming approach, created by NVIDIA (currently the dominant maker of GPUs specifically used for AI), that specifically lets developers actually write code that can properly run on and make full use of a GPU's own particular, specialized parallel architecture.

Here's the genuine problem CUDA actually solves. Writing code that correctly, efficiently splits a given task up across thousands of individual, simple parallel processing cores, and then correctly coordinates and combines all their many individual results back together again afterward, is genuinely a meaningfully different, more complex kind of programming challenge than writing regular, ordinary, sequential code intended for a genuinely conventional CPU. CUDA gives developers a genuinely structured, well-documented way of actually writing this kind of specifically parallel code, without needing to understand every single last low-level detail of the GPU's own actual physical hardware.

CUDA has become genuinely enormously significant and important throughout the whole AI industry, well beyond just being one single technical tool among several roughly interchangeable options, precisely because of how broadly and how deeply it's actually become adopted and embedded — most of the major AI frameworks (like PyTorch, which we mentioned earlier) are genuinely built with deep, extensive CUDA support, and a truly massive amount of the world's existing AI code and existing AI infrastructure genuinely, specifically assumes CUDA is actually available and genuinely being properly used. This is a real, significant part of why NVIDIA's own specific GPUs have become so genuinely dominant throughout the AI industry — it's not merely that their actual physical hardware happens to be particularly good (though it genuinely is), it's that CUDA, as a whole broader software platform and surrounding ecosystem, has become so genuinely deeply, widely embedded and integrated throughout the entire industry's software.

CUDA cores are the actual, individual, specific processing units that physically sit inside an NVIDIA GPU — they're genuinely the actual physical hardware components that CUDA, as the broader software platform we just discussed, is specifically designed to actually let a developer's code make proper, efficient use of.

Going back to our earlier CPU-versus-GPU comparison, CUDA cores are genuinely the specific, concrete embodiment of that "thousands of simple processing units" architectural idea we discussed earlier. A single modern, high-end NVIDIA GPU can genuinely contain many thousands of individual CUDA cores — sometimes even reaching into the range of several thousand to well over ten thousand individual cores, depending on the exact specific model and generation of the actual chip in question. Each single individual CUDA core, entirely on its own, is genuinely relatively simple and not all that powerful, but having thousands of them all genuinely, actually working together simultaneously, in parallel, is precisely what actually gives a GPU its whole overall enormous total computational power for exactly the kinds of parallel workloads we've been discussing throughout this whole explanation.

It's genuinely worth understanding that the raw, total number of CUDA cores a given GPU has isn't actually the complete, entire full picture of its overall real performance, all on its own — the actual speed and efficiency of each individual core, how quickly the whole chip can actually access its own memory (which we'll cover as its own separate, distinct topic shortly below), and other, additional specialized hardware features (like the tensor cores we'll also specifically cover shortly) all genuinely, meaningfully matter too. But

the total raw number of CUDA cores remains a genuinely useful, common, general, rough indicator of a given GPU's own overall raw parallel processing capability, and it's a genuinely commonly cited, referenced specification whenever people are actually comparing different specific GPU models against one another.

06 GPU Memory

GPU memory (often referred to specifically as VRAM, for "video RAM," reflecting graphics cards' own original historical purpose) refers to the actual, dedicated physical memory that sits directly on a given GPU itself, used specifically for storing the actual data the GPU is actually, currently working with — as genuinely distinct and separate from a regular computer's own main, separate system memory (RAM), which a CPU actually, instead, primarily uses for its own separate work.

Understanding GPU memory genuinely matters an enormous amount in practice, for a fairly simple, practical, concrete reason: an AI model, along with all of the actual data it's genuinely currently working with (like a batch of text currently being processed), needs to actually genuinely fit within a given GPU's own available memory capacity in order for that GPU to actually be able to properly work with and process it at all. Modern large language models can genuinely be enormous — some of the largest ones out there require many tens, or even hundreds, of gigabytes of memory space just to actually hold the model's own learned parameters alone, well before you even get to accounting for any of the actual additional working data needed for actually running it. If a given model genuinely doesn't actually fit within a single GPU's own available memory, you either genuinely need a GPU that actually has considerably more memory capacity available, or you genuinely need to actually split the given model up cleverly across multiple different separate GPUs working together (a genuinely real, meaningful technical challenge in its own right, connecting quite directly to the NCCL topic we'll get to shortly below).

GPU memory capacity has actually become one of the single most significant, important, and genuinely often limiting practical factors in modern AI infrastructure, precisely because of how large today's models have genuinely grown to actually become. A GPU with genuinely excellent raw processing speed, but genuinely insufficient available memory capacity, simply, genuinely can't actually run a given particular large model at all — no matter how fast that GPU's own individual cores might otherwise genuinely, individually be. This is exactly why, when people actually compare and choose between different specific GPU models for genuine, serious AI work, memory capacity is very often one of the single most heavily weighted, important considered factors, right alongside raw computational processing speed itself.

07 Tensor Cores

Tensor cores are a further, more specialized type of processing unit found specifically on newer NVIDIA GPUs, genuinely distinct and separate from the more general-purpose CUDA cores we already discussed above — and they're specifically, deliberately built to accelerate one particular, very specific type of mathematical operation that happens to be especially, genuinely central and important to how AI models actually work.

That specific particular operation is matrix multiplication — which we already actually briefly mentioned earlier, back in the GPU Architecture section, as being genuinely the fundamental, core mathematical operation underlying almost everything a neural network actually, fundamentally does. While regular CUDA cores are already genuinely quite good at handling this particular kind of calculation, tensor cores are even

further, additionally specialized specifically and exclusively for it — they're genuinely, specifically built to actually perform certain particular specific kinds of matrix math considerably faster than even regular CUDA cores alone genuinely, actually could.

This specific additional specialization genuinely, actually matters an enormous amount for real-world, practical AI performance. NVIDIA has reported that tensor cores can genuinely provide dramatically faster performance, for exactly the specific kinds of calculations that AI models actually specifically, heavily rely on, compared to using regular CUDA cores alone, entirely on their own, for that exact same specific kind of underlying work. This is a genuinely excellent, clear, concrete illustration of a fairly broader, more general important principle in modern computer hardware design: rather than only ever building genuinely more and more general-purpose processing power, chip designers increasingly, deliberately build specific, dedicated hardware specifically optimized for particular, especially common and especially important specific types of calculations — and given precisely how absolutely central matrix multiplication genuinely, specifically is to essentially all of modern AI, tensor cores genuinely represent a particularly clear, well-motivated, sensible example of exactly this same broader specialization principle actually, concretely playing out in real, practical, real-world hardware.

08 NVIDIA Driver Stack

A driver, as a completely general software concept, is the specific piece of software that actually lets a computer's own operating system properly communicate with and correctly control a given specific piece of physical hardware. The NVIDIA driver stack refers specifically to the actual collection of software layers that genuinely, actually let a computer properly recognize and correctly, properly make use of an installed NVIDIA GPU in the first place.

Without a genuinely properly installed and correctly, properly functioning driver, an operating system genuinely, actually has no reliable, correct way of actually knowing how to properly communicate with a given specific physical GPU at all — the actual physical hardware chip might genuinely be correctly, physically installed and present inside a given machine, but without the correct, matching driver software genuinely also properly installed alongside it, the rest of the overall computer system genuinely, actually can't correctly, actually make any real, meaningful use of it whatsoever. The driver is what actually genuinely handles the lowest-level, most fundamental communication with the actual physical hardware itself — things like actually correctly sending genuine, real computational work to the GPU to be properly processed, and then actually correctly, properly retrieving the genuine, real resulting results back once that work has actually, genuinely finished being properly completed.

For anyone genuinely, actually setting up AI infrastructure themselves in practice, getting the correct, matching driver version genuinely, properly installed correctly is often one of the single most common, genuinely frustrating early practical setup and configuration challenges people actually, genuinely run into. Different specific driver versions genuinely need to actually, properly match up correctly with the other various different specific software layers sitting above them in this whole stack (like the CUDA toolkit, which we'll actually cover next, right below) — a genuine, real mismatch between these different specific software layers' own respective, particular versions is a genuinely extremely common, well-known, and often quite frustrating source of real technical problems specifically when actually, practically trying to get a given GPU-based AI system properly, correctly, and reliably up and running in the first place.

The CUDA toolkit is the genuinely complete collection of actual software tools, code libraries, and documentation that NVIDIA actually, genuinely provides specifically to let developers actually write, properly build, and correctly run genuine CUDA-based code — it sits as a genuinely distinct software layer, one level up above the actual raw driver we just discussed, specifically providing the higher-level tools a developer actually, genuinely needs to properly do real, actual, hands-on AI development work.

Where the driver, as we just discussed, specifically handles the most fundamental, lowest-level communication between the operating system and the actual physical hardware chip itself, the CUDA toolkit specifically provides the genuinely higher-level tools a developer actually, genuinely uses day to day — things like a genuine compiler (which is specifically the tool that actually translates a developer's own human-written code into the actual specific low-level instructions a GPU can genuinely, actually understand and correctly execute), and a whole range of genuine, various pre-built code libraries that already, genuinely handle a lot of the common, genuinely repeated, everyday underlying computational work, so that individual developers genuinely, actually don't each need to separately write all of that same underlying complex code entirely themselves, completely from total scratch, every single time.

For most everyday developers who are actually just using a genuinely higher-level framework like PyTorch (which we already briefly mentioned earlier, back at the very start of this whole explanation), a lot of the actual CUDA toolkit's own underlying complexity genuinely, actually stays fairly comfortably hidden and abstracted away underneath — a given framework like PyTorch itself already, genuinely handles most of the actual direct interaction with CUDA on a developer's own behalf, so a typical developer genuinely doesn't usually, actually need to write raw, direct CUDA code themselves, entirely by hand. But having the genuinely correct, properly matching version of the CUDA toolkit actually, properly installed underneath everything else remains genuinely essential regardless, since these genuinely higher-level frameworks are themselves actually, ultimately built directly right on top of it.

cuDNN (which specifically stands for "CUDA Deep Neural Network library") is yet a further, additional specialized software library, built specifically, deliberately on top of the more general CUDA toolkit we just discussed, and it's specifically, deliberately optimized for the particular, specific kinds of calculations that genuinely, specifically show up repeatedly throughout deep learning and neural networks in particular — rather than the more broadly general, wider range of parallel computing tasks that the more general CUDA toolkit itself is actually, more broadly designed to genuinely support.

Think of the relationship here as being somewhat similar in overall spirit to the relationship between CUDA cores and tensor cores that we already discussed earlier — CUDA, as the more general underlying toolkit, genuinely already lets you actually do general-purpose parallel computing work of many different, varied kinds, while cuDNN specifically, further, additionally optimizes for the particular, specific operations that genuinely, specifically come up over and over again in deep learning work specifically — things like the particular specific mathematical operations genuinely involved in a neural network's own internal individual layers.

The genuine practical value cuDNN actually, genuinely provides is real, meaningful additional speed, specifically for exactly these particular deep-learning-specific operations, well beyond what you'd actually get from simply using the more general CUDA toolkit alone, entirely on its own, without cuDNN's own further, additional specific optimizations layered properly on top of it. Just like with the CUDA toolkit itself, most

everyday individual developers genuinely, actually don't interact with cuDNN directly themselves, by hand — it instead, genuinely operates quietly underneath the higher-level frameworks like PyTorch that a typical developer actually, genuinely, directly works with day to day — but its own specific presence, and having the genuinely correct, properly matching version actually, properly installed underneath everything else, meaningfully, genuinely affects how fast a given model actually, genuinely trains and actually genuinely runs in real, practical, everyday practice.

11 NCCL

NCCL (pronounced "nickel," and specifically standing for "NVIDIA Collective Communications Library") specifically addresses a genuinely real, important, practical problem that naturally, genuinely comes up once you actually need to use multiple separate GPUs together, working properly, closely, and reliably in real coordination with one another — which, given how enormous today's largest AI models have genuinely, actually become (connecting directly back to the GPU memory discussion earlier above), is genuinely an extremely common, everyday, and genuinely important real-world practical scenario.

When you're actually, genuinely training or running a given AI model across several separate GPUs at once, working properly together, those different individual GPUs genuinely, actually need to be able to efficiently, reliably communicate with each other — for example, actually, genuinely sharing and properly, correctly combining their own individual, separate partial results together, so that the whole overall combined system can actually, genuinely arrive at one single, unified, correct, final overall answer. NCCL is specifically, deliberately designed to actually, genuinely handle exactly this particular kind of communication as efficiently and as reliably as reasonably, genuinely possible — providing developers with a set of well-designed, pre-built, ready-to-use tools specifically for actually, reliably coordinating and properly sharing data across multiple different GPUs, without each individual developer separately, individually needing to write all of that same underlying complex multi-GPU coordination logic entirely by hand themselves, completely from total scratch, every single time.

This particular capability genuinely, actually becomes increasingly critical and important the larger a given specific AI workload actually, genuinely gets. Training one of today's genuinely largest modern language models often actually genuinely requires many hundreds, or sometimes literally even many thousands, of individual GPUs, all genuinely working together properly, closely, and reliably in real, actual coordination — and the genuine overall efficiency of exactly how well those many separate individual GPUs can actually, genuinely communicate with each other becomes a genuinely real, significant, and often quite substantial factor in determining the whole overall system's own genuine total real-world practical performance, right alongside the genuine raw individual computational power of each individual GPU considered entirely separately, all on its own.

12 GPU Scheduling

GPU scheduling refers to the genuinely real, important, practical process of actually deciding which specific task or which specific job genuinely, actually gets to actually use a given available GPU, and genuinely, actually exactly when — this genuinely becomes an especially real, important, and often quite significant practical concern specifically in any kind of shared environment, where multiple genuinely different users, or multiple genuinely different teams, or multiple genuinely different separate applications, all genuinely need to actually, meaningfully share access to some kind of overall limited, shared pool of available GPU resources together.

Here's the genuine, real underlying practical problem GPU scheduling actually, genuinely needs to properly solve. GPUs, particularly genuinely powerful, high-end ones specifically well-suited to serious, real AI work, remain genuinely quite expensive pieces of specialized physical hardware, and organizations genuinely, typically, sensibly don't actually want to simply, wastefully buy an entirely completely separate, dedicated GPU for literally every single individual team or every single individual specific project they might genuinely happen to have running internally. Instead, a genuinely shared pool of available GPUs is typically actually, sensibly maintained together, and a genuinely dedicated scheduling system specifically, actively decides which particular specific job actually, genuinely gets to actually use which particular specific available GPU, and for genuinely exactly how long, in a genuinely fair, efficient, and sensible overall way.

Good GPU scheduling genuinely, actually needs to properly, sensibly balance several genuinely real, distinct, competing considerations together at once. It genuinely, actually needs to try to keep the whole overall available pool of GPUs genuinely, actually as busy and as fully, efficiently utilized as reasonably possible (since an idle, unused, genuinely expensive GPU sitting there completely unused represents real, genuine, unnecessarily wasted money, sitting there entirely idle). It genuinely, actually needs to try to be reasonably, appropriately fair across the whole organization's own various different competing users and teams, rather than genuinely letting just any one single particular team or user consistently, unfairly monopolize the entire whole shared available pool of resources all to themselves alone. And it genuinely, actually needs to properly, sensibly handle jobs of genuinely different, varying relative priority appropriately — for example, appropriately letting a genuinely urgent, time-sensitive production request reasonably, sensibly take real priority over a genuinely much less time-sensitive, more experimental research job that could, quite reasonably, genuinely just as easily wait its own proper turn a while longer instead. This connects quite directly and closely back to the rate limiting and cost allocation concepts we already, previously covered back in the earlier Week 10 enterprise architecture material — GPU scheduling is really, genuinely the same broader, fundamental underlying idea of fairly, sensibly, and efficiently sharing a genuinely limited, valuable, shared resource across many different competing internal consumers, just now specifically, particularly applied here directly to actual, real physical computing hardware itself, rather than to the more purely software-based API rate limits we had already originally, specifically discussed back in that earlier explanation.

Production AI Serving

The Big Picture First

In the last explanation, we covered the raw hardware — GPUs, CUDA, memory. But having powerful hardware alone doesn't automatically mean you can serve an AI model efficiently to lots of real users at once. Here's the specific problem this whole section addresses: running a language model for just yourself, one question at a time, is relatively straightforward — but running that same model in production, handling potentially thousands of simultaneous users, each sending requests of wildly different lengths, all while trying to keep the GPU's expensive hardware genuinely busy rather than sitting idle, is a genuinely much harder engineering problem. Production AI serving is the whole discipline built around solving exactly this problem — a collection of specialized software tools and clever techniques specifically designed to squeeze the maximum possible useful work out of expensive GPU hardware while serving real, live traffic. Let's work through it.

01 vLLM

We actually already introduced vLLM briefly, back in the very first LLM APIs explanation, as a specialized tool for serving models efficiently. It's worth revisiting here in much more depth, because it's genuinely one of the most influential and widely-used tools in this entire space.

vLLM's single biggest, defining innovation is something called PagedAttention. To understand why it matters, you need to understand a genuine, real problem it was specifically built to solve: when a model is actually processing a request, it needs to keep track of a growing amount of intermediate data (something we'll cover shortly as the "KV cache"), and older, more naive serving approaches would typically reserve one single large, continuous, unbroken chunk of GPU memory for this per request — similar to trying to always park one single, entire, unbroken city block just for one specific car, even though that car might only actually need a small handful of individual parking spaces. This wastes an enormous amount of genuinely valuable, expensive GPU memory. PagedAttention instead borrows an idea from how regular computer operating systems manage memory more generally — breaking this data up into smaller, more flexible chunks (or "pages") that can actually be allocated and freed up much more efficiently, similar to actually assigning individual parking spaces to a car as it genuinely needs them, rather than reserving one whole giant block upfront regardless of actual need.

The genuine, practical, real-world payoff of this approach is substantial — vLLM has been shown to achieve meaningfully higher throughput and better GPU memory efficiency compared to older, more naive serving approaches, letting a given GPU actually serve considerably more simultaneous users at once from the exact same underlying hardware. Because of this genuine efficiency, combined with the fact that it's open-source, free, and reasonably straightforward to actually get started with (you can often start serving a model pulled directly from Hugging Face with comparatively little setup effort), vLLM has become something of a widely-adopted default, general-purpose choice across the industry for serving open-source language models in production.

02 TensorRT

TensorRT is NVIDIA's own general-purpose toolkit specifically for optimizing and actually running trained AI models as fast as genuinely, technically possible on NVIDIA's own specific GPU hardware — it's worth understanding as a broader, more general foundation, distinct from TensorRT-LLM (which we'll cover next, and which is specifically built on top of it, for language models specifically).

The core underlying idea behind TensorRT is that a model, once it's already fully trained, can actually be further optimized quite considerably, specifically for the particular exact hardware it's actually going to be run on — techniques like combining several separate individual mathematical operations together into one single, more efficient, combined operation, or reducing the numerical precision used for certain particular calculations where that reduction doesn't meaningfully hurt the actual final result's quality (which connects directly to the model quantization topic we'll cover later below). TensorRT specifically takes an already-trained model and effectively rebuilds it into a specifically optimized, hardware-specific format designed to run considerably faster on NVIDIA's own particular chips.

The genuine tradeoff worth understanding clearly here is that this deep, hardware-specific optimization process genuinely comes at a real cost in flexibility and setup effort. Building one of these specifically optimized "engines," as they're generally called, genuinely requires real, deliberate, careful upfront configuration work, and the resulting optimized engine ends up genuinely tied quite specifically and closely to the particular exact hardware and particular exact model configuration it was actually built for — meaning it's genuinely less of a quick, flexible, "just point it at a model and immediately go" kind of tool, compared to something like vLLM, and considerably more of a tool for squeezing out that final, additional degree of maximum possible raw performance once a team has already genuinely settled on their exact specific model and exact specific hardware setup, and is willing to actually, genuinely invest real, meaningful additional setup time and effort to get there.

03 TensorRT-LLM

TensorRT-LLM is NVIDIA's own further, more specialized version of TensorRT, specifically built and tailored for actually serving large language models in particular, rather than covering the considerably broader, more general range of different AI model types that the base TensorRT toolkit itself is more generally designed to actually handle.

Building directly on top of everything TensorRT already provides, TensorRT-LLM specifically adds in a whole range of additional optimizations genuinely specific to how language models actually work — things like specialized, further-optimized handling of the KV cache we'll discuss shortly below, and specific further support for some of the more advanced serving techniques we'll also cover later in this same explanation, like speculative decoding. The general, overall goal remains fundamentally the same as with the base TensorRT toolkit itself: squeeze out the genuine, absolute maximum possible raw performance specifically from NVIDIA's own particular hardware, now specifically tailored and focused on language models in particular.

In practice, TensorRT-LLM is generally considered the choice for teams that are genuinely fully, deliberately committed to NVIDIA's hardware specifically, and that genuinely have both the real, meaningful technical expertise and the real, meaningful time available to actually properly invest in its more involved setup process — reports suggest this initial setup can genuinely take something like one to two full weeks of dedicated work, compared to the considerably faster, more immediate path to getting started that a tool like vLLM typically genuinely offers. The genuine payoff for that real additional upfront investment is typically the

very best possible raw performance actually achievable specifically on NVIDIA's own hardware — but it does genuinely come at the real cost of that meaningfully greater setup complexity, and a real, genuine, tighter lock-in specifically to NVIDIA's own particular hardware and software ecosystem as a whole.

04 Ollama

We actually already covered Ollama in real detail, back in the very first LLM APIs explanation, specifically as a tool for running AI models locally on your own personal computer. It's genuinely worth briefly revisiting here, specifically to clarify how it fits properly into this whole broader serving landscape, alongside the various other, more production-oriented serving tools we're covering throughout this whole explanation.

Where the other tools covered throughout this whole explanation (vLLM, TensorRT-LLM, and the others we're about to get to) are all specifically built and optimized for serving many simultaneous users efficiently in real, live production, Ollama's own particular design focus remains fundamentally different — it's specifically, deliberately optimized for ease of use and genuine simplicity, on a genuinely single individual personal machine, rather than for maximizing raw genuine throughput across many simultaneous users at real, meaningful production scale. This makes Ollama a genuinely excellent, well-suited choice specifically for local individual development, experimentation, and genuinely personal individual use, but it's generally, genuinely not the right, appropriate tool specifically for actually serving a genuine, real production application handling many real, simultaneous concurrent users at once.

Understanding this particular distinction clearly genuinely matters, precisely because it's a fairly common, easy point of real confusion for beginners specifically getting properly started in this whole broader space — the fact that a given tool is genuinely excellent and well-suited for one entirely particular specific use case (like Ollama genuinely is, specifically for local individual development) doesn't necessarily, automatically mean it's also genuinely the right, correct tool for an entirely different, separate specific use case (like actually serving real production traffic at genuine, meaningful scale), even though both of these particular use cases genuinely do, on the surface, broadly, loosely involve "running an AI model" in some fairly general, broad overall sense.

05 Hugging Face TGI

TGI (Text Generation Inference) is Hugging Face's own dedicated production serving framework, and it's genuinely worth understanding both its real, significant historical importance to this whole broader field, and, importantly, its genuinely current actual status as of right now.

Historically, TGI was genuinely one of the very first widely-adopted dedicated serving engines specifically built for language models, and it genuinely played a real, significant, important role in actually powering some of Hugging Face's own major, widely-used products, while also genuinely helping push the whole broader industry forward toward much better-optimized ways of actually serving these kinds of models efficiently, well beyond simply, naively calling a model directly in raw Python code. It integrates especially closely and tightly with the very broad, extensive Hugging Face ecosystem of models more generally, making it a genuinely natural, comfortable fit specifically for teams that are already, genuinely working extensively within that particular same ecosystem.

However, it's genuinely important to be upfront and honest here about a real, significant, current development: as of quite recently, Hugging Face has actually put TGI into what's specifically called "maintenance mode" — meaning it now generally only genuinely receives bug fixes going forward, rather than continuing to receive genuinely significant, meaningful new features and improvements as it did

previously. Hugging Face itself has actually, publicly recommended that teams instead move toward using either vLLM or SGLang (which we'll cover just next, right below) going forward, and Hugging Face's own Inference Endpoints product has itself actually already switched over to defaulting to vLLM. This particular detail genuinely, concretely illustrates something worth keeping in mind throughout this entire fast-moving space more generally — specific tools and specific frameworks genuinely, actively rise and fall in relative prominence and relative recommendation status fairly quickly, so it's genuinely always worth checking current, up-to-date, recent sources before actually committing your own team to any one single particular specific tool for any genuinely serious, real production use.

06 SGLang

SGLang is a considerably newer serving framework that has actually, genuinely risen to real prominence fairly quickly and fairly recently, specifically built around a core technique called RadixAttention, which is genuinely particularly, especially well-suited to a specific category of workload that's become increasingly, genuinely common and important: multi-turn conversations, RAG-based applications, and genuine AI agents — all cases where a considerable amount of the actual underlying prompt content genuinely, meaningfully gets reused, in whole or in significant part, again and again across multiple separate, successive requests.

Here's the core, genuine underlying insight behind RadixAttention. In a genuinely typical multi-turn chat conversation, or in a genuine agent that's repeatedly calling tools and continuing to build on the exact same underlying growing context, a very considerable, large portion of a given new incoming request's actual full prompt is often genuinely, literally identical to what a given system has already, previously processed just moments before, in an earlier immediately preceding turn. RadixAttention specifically, deliberately, cleverly caches this genuinely already-processed shared computation, so the given system genuinely doesn't ever have to actually redo that same exact already-completed work all over again from scratch, each and every single subsequent time. Reported real-world benchmark results suggest SGLang can genuinely, meaningfully outperform vLLM by a real, quite significant margin specifically on exactly these kinds of shared-context, conversational-style workloads — with some reported real benchmarks showing genuinely dramatically higher throughput and genuinely, considerably lower latency specifically for these kinds of repeated-context use cases in particular.

The genuinely practical overall takeaway here is that SGLang has become an especially, particularly strong, well-suited specific choice specifically for exactly the kinds of genuinely conversational, RAG-based, and agent-based applications we've actually covered at considerable, real length throughout this entire broader series — precisely the specific kinds of applications where a meaningful, substantial portion of the given underlying prompt content genuinely does tend to actually repeat quite considerably, again and again, across many successive requests, in a way that a more general-purpose serving engine might genuinely, otherwise simply, unnecessarily end up needlessly, wastefully redoing all over again, entirely from complete scratch, every single time.

07 LMDeploy

LMDeploy is a specialized model-serving and model-compression toolkit that comes from the InternLM ecosystem, and its own particular core engine is specifically called TurboMind. Its own particular, specific design focus centers especially heavily around achieving genuinely maximum possible raw throughput per

individual GPU, especially and particularly specifically when working together in combination with quantized models (connecting quite directly to the model quantization topic we'll actually cover shortly, later below, in this same explanation).

LMDeploy's own particular, specific combination of techniques — including its own persistent batching approach, and its own specific, particular blocked approach to KV cache management (connecting directly to the KV cache topic we'll also cover shortly below) — has reportedly, genuinely allowed it to achieve meaningfully, considerably higher raw request throughput compared to vLLM in certain specific, particular benchmark comparisons, especially and particularly specifically for 4-bit quantized versions of certain popular, well-known specific model families.

LMDeploy genuinely tends to be a particularly well-suited, strong specific choice for teams that are genuinely, specifically prioritizing squeezing the genuine, absolute maximum possible raw efficiency specifically out of quantized models in particular — meaning models that have already been deliberately, specifically compressed down to use meaningfully, considerably less raw memory and meaningfully less raw computation than their genuine, original full-precision counterpart versions, at the cost of only some genuinely quite small, typically fairly minor, and often genuinely well worth it corresponding real reduction in overall raw output quality. Like several of the other more specialized tools we've covered throughout this whole explanation, it genuinely represents a real, meaningful, particular specific optimization tradeoff — trading off some genuine additional setup complexity and some genuine additional specificity of its own particular focus, specifically in exchange for genuinely, meaningfully superior raw real-world performance within that own particular, specific focused use case.

08 Continuous Batching

Continuous batching (which you may also sometimes genuinely see referred to elsewhere as "inflight batching") is one of the single most genuinely important underlying techniques that actually makes efficient, modern production LLM serving genuinely possible in the first place — and understanding it clearly genuinely helps make sense of why several of the specific tools we've already covered throughout this whole explanation are actually so meaningfully, considerably faster than a more naive, simple, unoptimized serving approach would otherwise genuinely be.

To understand why continuous batching genuinely matters so much, it helps to first understand the real, genuine problem it actually, specifically solves. GPUs, as we already discussed at real length in the previous explanation, are genuinely, fundamentally at their own particular best specifically when they're actually doing the exact same kind of calculation on many different, separate pieces of data all together at once, in parallel — this general underlying idea, when specifically applied to genuinely serving many separate individual user requests together, is generally referred to as "batching." But here's the genuine, real practical complication: different individual user requests naturally, genuinely arrive at meaningfully different times, and they each genuinely, naturally require meaningfully different amounts of actual total processing time to fully, completely finish (since some given individual requests genuinely, naturally want considerably longer overall responses than others do). A genuinely more naive, simple, older batching approach would typically wait around to actually collect a genuinely full batch of separate individual requests together first, and would then genuinely, correspondingly have to wait for literally every single individual request within that entire given batch to fully, completely finish, before it could then finally actually go on to start any genuinely new, fresh batch of requests at all.

Continuous batching instead genuinely, cleverly allows genuinely new incoming individual requests to actually, dynamically join an already actively in-progress, ongoing batch, and correspondingly also allows individual requests that have already, genuinely finished to actually leave that same ongoing batch and immediately free up their own space, all without the given overall system ever needing to actually wait around for the entire whole batch, considered together, to fully finish all together, all at once, in perfect lockstep. This genuinely keeps the GPU considerably, meaningfully more consistently, continuously busy overall, since it genuinely never has to sit around uselessly waiting on just one single particular slow, long individual request to fully finish before it can genuinely go on to actually pick up and start any brand new incoming work. This particular technique genuinely represents one of the single most significant overall reasons why modern, dedicated serving engines like vLLM are genuinely so considerably, dramatically more efficient in practice than simply, naively calling a model directly, one individual request entirely at a time, in a purely sequential, one-after-another fashion.

09 KV Cache

The KV cache (short for "key-value cache") refers to intermediate data that a language model genuinely needs to actually keep track of and properly store while it's actually in the process of generating a given response, and understanding it clearly genuinely helps explain why GPU memory (which we discussed at real length in the previous explanation) becomes such a genuinely critical, real, and often quite significant limiting practical factor specifically when actually, genuinely serving language models in real production.

Without getting too deeply into the underlying technical mathematics here, the genuine, core underlying idea is this: as a model actually, genuinely generates a response, one individual token (a small individual word-like piece of text) at a genuine time, it needs to actually keep genuinely referring back to everything it has already, previously seen and generated so far, in order to properly, correctly decide on the actual next appropriate token to generate. Rather than needlessly, wastefully recalculating this exact same underlying relevant information completely again from total scratch for literally every single new individual token it happens to generate, the model instead genuinely, cleverly caches (meaning it actually genuinely stores) certain particular specific intermediate values, so it can then genuinely just efficiently reuse them again immediately, rather than ever needing to needlessly redo that exact same underlying calculation work all over again.

This particular caching approach genuinely, meaningfully speeds things up considerably — but it also genuinely, correspondingly consumes real, actual, meaningful GPU memory, and critically, the total overall size of this particular cache genuinely, actually grows the longer a given ongoing conversation or a given individual response actually genuinely becomes. This is exactly, precisely the specific underlying real problem that vLLM's own PagedAttention technique, which we already actually discussed above, was specifically, deliberately built to actually help solve — managing this particular growing cache considerably more efficiently, so a given available GPU can genuinely, actually serve meaningfully more simultaneous concurrent users at once, without needlessly, wastefully running out of its own genuinely available memory. Understanding the KV cache clearly genuinely helps explain why longer conversations and genuinely longer individual responses meaningfully, correspondingly cost real, more money and genuinely take real, more memory to actually properly serve, compared to considerably shorter, more brief individual exchanges.

10 Speculative Decoding

Speculative decoding is a genuinely clever technique specifically designed to actually speed up how fast a model can genuinely generate its own given response, and the core underlying idea here is genuinely worth understanding well, since it's a fairly clever, non-obvious real approach.

Here's the core, genuine underlying idea. Normally, a genuinely large, powerful, and fairly slow language model has to actually generate its response one individual single token entirely at a time, with each new individual token genuinely, fully depending directly on all of the ones that actually already came immediately before it. Speculative decoding instead genuinely uses a considerably smaller, meaningfully faster "draft" model to actually quickly generate several individual candidate next tokens all at once, all together, and then has the larger, considerably more capable, "real" primary model genuinely just verify all of those particular candidate tokens together, all at once, in a genuinely single verification pass, rather than the larger model ever needing to actually, individually, separately generate every single one of those tokens completely by itself, entirely from complete scratch, one at a genuinely time.

The genuine, real payoff here comes specifically from the fact that verifying a whole given set of already-proposed candidate tokens together, all at once, in one single combined pass, is genuinely considerably faster overall than the larger, primary model actually having to individually, separately generate that exact same equivalent number of tokens completely on its own, entirely from scratch, one at a genuinely, painstaking time. When the smaller, faster draft model's own proposed candidate guesses genuinely happen to actually turn out correct (which, for many fairly common, everyday, predictable pieces of text, does genuinely happen quite frequently and quite often in practice), the given overall system genuinely gets a meaningful, real speed boost, essentially entirely for free. When the smaller draft model's own particular guesses genuinely happen to turn out wrong instead, the larger, primary model simply falls back to properly, correctly generating that particular specific token itself, entirely on its own, exactly as it always genuinely, normally would have anyway — meaning this particular technique genuinely never actually risks any real reduction whatsoever in the given final response's own overall quality, it only genuinely, potentially ever helps speed things up.

11 Model Quantization

We've now touched on model quantization several separate times throughout this whole explanation, so it's genuinely worth properly covering it here as its own fully dedicated, complete topic. Quantization refers to the general technique of actually reducing the numerical precision genuinely used to actually store a given model's own internal learned parameters — meaning representing each individual internal number using genuinely fewer actual bits of raw data than the model's own genuine, original full precision would normally otherwise actually require.

To understand this idea clearly and intuitively, think of it somewhat like this: a model's own internal parameters are typically originally, genuinely stored using a fairly high degree of raw numerical precision (meaning many actual decimal places of genuine numerical accuracy). Quantization genuinely, deliberately reduces that overall precision — for example, converting from a genuinely higher-precision original format down to a meaningfully lower-precision format that uses considerably, meaningfully fewer actual bits per individual stored number. This is genuinely somewhat analogous to rounding a genuinely long, complex, precise decimal number like 3.14159265 down to simply, roughly 3.14 instead — you genuinely do lose some real, small degree of precise underlying precision in doing so, but the overall resulting number genuinely still remains quite close to, and quite usably, practically representative of, the actual real original value.

The genuine, real payoff here is considerable and quite significant: a genuinely quantized model takes up considerably less actual raw GPU memory space, and it can genuinely often actually run meaningfully, considerably faster too, since there's genuinely, correspondingly less actual raw data that needs to be moved around and genuinely processed. The genuine tradeoff, correspondingly, is some real, small potential reduction in the given model's own overall output quality — though, genuinely, well-designed quantization approaches can often actually genuinely achieve quite considerable, meaningful real memory and speed savings while only incurring a genuinely quite small, often barely noticeable corresponding real reduction in overall practical output quality. This is exactly, precisely why quantization has genuinely become such a widely, commonly used technique throughout this whole broader field — it lets teams genuinely run meaningfully larger, more capable models on genuinely more modest, more affordable available hardware, or genuinely serve considerably more simultaneous users at once from that exact same given underlying hardware, all in direct exchange for only a genuinely quite small, often well worth it corresponding tradeoff in the model's own overall raw output precision.

12 AI Serving Optimization

To genuinely close out this whole explanation, it's worth pulling together the various individual specific techniques we've actually covered throughout this entire explanation into one final, unified, overall picture of what genuinely, practically effective AI serving optimization actually, genuinely looks like altogether, in real, genuine, holistic combined practice.

Good AI serving optimization generally, genuinely means thoughtfully combining several distinct techniques together, rather than ever genuinely relying on just any single one of them entirely alone, all on its own.

Choosing the genuinely right underlying serving engine for your own particular specific given workload matters enormously — as we've genuinely seen quite clearly throughout this whole explanation, vLLM genuinely offers a strong, solid, well-rounded general-purpose default; SGLang genuinely particularly, specifically excels at conversational and genuinely agent-based workloads specifically; and TensorRT-LLM genuinely offers the very best possible raw performance specifically for teams genuinely fully, deliberately committed to NVIDIA hardware and genuinely willing to actually properly invest the real, additional required setup effort. **Applying quantization** where it genuinely, reasonably makes real practical sense can genuinely meaningfully reduce both cost and improve overall speed, often quite considerably, in direct exchange for only some genuinely quite small, often well worth it corresponding tradeoff in output precision. **Using continuous batching and efficient KV cache management** genuinely helps ensure the given underlying GPU stays as consistently, genuinely busy and as efficiently, productively utilized as reasonably possible, rather than ever wastefully sitting around idle. And **considering speculative decoding**, where it genuinely, reasonably applies well, can genuinely help meaningfully speed up actual response generation, all without ever actually, genuinely sacrificing any real degree of overall output quality whatsoever in doing so.

The genuinely important, broader overall takeaway to end on here, and really across this whole entire Week 12 material as a genuine, complete whole, is that serving an AI model efficiently, genuinely well, at real, meaningful production scale, is a genuinely real, distinct engineering discipline entirely all its own — every bit as genuinely real and important, and every bit as genuinely worth taking real, careful, deliberate time to properly, thoroughly learn and understand well, as the actual prompting, the actual RAG, and the actual agent-building work we've genuinely already covered together at such real, considerable length throughout this whole entire broader course. A genuinely brilliant, well-designed AI application, even one that's genuinely built with real, considerable, careful skill and real thoughtful care throughout, can still genuinely end up feeling frustratingly slow, and can still genuinely end up costing considerably, unnecessarily more

than it really, genuinely needs to, if the actual underlying serving infrastructure quietly sitting there underneath it hasn't itself also actually, genuinely been properly, carefully, thoughtfully optimized right alongside it.

Distributed AI Infrastructure

The Big Picture First

Everything we've covered across this whole Week 12 material so far has largely assumed one GPU, or maybe a handful of them, working together on one machine. But the largest, most demanding AI workloads — training a genuinely huge model, or serving a genuinely massive volume of live traffic — routinely require coordinating work across dozens, hundreds, or even thousands of separate machines, each with their own GPUs, all working together as one unified whole. This introduces a genuinely different category of engineering challenge: how do you write code that can actually run across many separate machines at once, how do you reliably keep track of and manage all of that underlying hardware, and how do you make sure the whole system gracefully scales up when demand increases, and scales back down again when it doesn't, all without constant manual intervention. This final section covers the tools and concepts built specifically to solve exactly this class of problem.

01 Ray Fundamentals

Ray is a distributed computing framework, and the single most important thing to understand about it is the specific problem it's actually solving: normally, writing code that correctly, reliably splits work across many separate machines is a genuinely hard, specialized kind of programming, requiring real expertise in distributed systems that most everyday developers, understandably, simply don't have. Ray's whole core promise is to let a developer write largely ordinary-looking Python code, and have Ray itself handle the genuinely hard, complicated underlying work of actually distributing that code's execution across a whole cluster of many separate machines.

The way this actually works in practice is refreshingly simple on the surface. A developer takes a regular Python function and adds a small marker to it (specifically, something called a decorator) that tells Ray "this particular function can be run remotely, out on the broader distributed cluster, rather than only locally, right here." From that point forward, Ray itself takes care of the genuinely hard parts — figuring out which specific available machine should actually run a given particular piece of work, actually sending that work out to it, and then reliably collecting the resulting results back once that work has actually finished. This lets a developer write code that can genuinely scale from running on their own single personal laptop, all the way up to running across a genuinely large cluster of many machines, without needing to fundamentally rewrite that same underlying code each time.

Ray isn't specifically or exclusively built only for AI workloads in some narrow sense — it's a genuinely general-purpose distributed Python framework. But it's become especially, particularly popular and important specifically within the AI world precisely because so much of modern AI development is already itself written in Python, and because AI workloads (training a model, or serving many simultaneous requests) are exactly the kind of work that genuinely, naturally benefits enormously from being properly, efficiently spread across many separate machines working together. Ray Serve and Ray Train, which we'll cover next, are Ray's own specific, dedicated libraries built directly on top of this same core general foundation, specifically tailored for AI-specific work.

02 Ray Serve

Ray Serve is Ray's own dedicated library specifically for actually serving AI models in production — taking the general underlying distributed computing capability that Ray provides more broadly, and specifically packaging it into a tool purpose-built for exactly the serving challenges we discussed in the previous explanation.

The genuine value Ray Serve specifically brings comes largely from flexibility, combined with genuinely proper, native distributed scaling built directly in from the ground up. Since it's built directly on top of Ray's own general-purpose distributed framework, Ray Serve can genuinely, naturally spread a given model's serving workload across many separate machines, and can genuinely support fairly sophisticated, flexible serving patterns — for example, actually running several different, separate models together as part of one single combined pipeline, or actually running custom, stateful serving logic involving several different cooperating components, in ways that some of the more narrowly-focused, single-purpose serving engines we covered in the previous explanation (like vLLM or TensorRT-LLM) genuinely aren't quite as naturally, flexibly well-suited to handling on their own.

In practice, Ray Serve is often actually used together with, rather than strictly instead of, some of the specialized serving engines we already covered in the previous explanation — for example, using Ray Serve specifically to handle the overall broader orchestration and distributed scaling of a whole given serving pipeline, while actually calling out to something like vLLM underneath, specifically to handle the genuinely raw, actual token-generation work itself, as efficiently as reasonably possible. This combination genuinely lets a team get the particular specific benefits of both tools together at once — vLLM's own particular deep serving-specific optimizations, combined with Ray Serve's own broader, more flexible distributed orchestration and scaling capabilities, layered properly on top.

03 Ray Train

Ray Train is Ray's own other major dedicated AI-specific library, this one specifically focused on distributed training rather than serving — helping a team actually train a given model across many separate machines and many separate GPUs working together, rather than being limited to training only on one single machine alone.

Training a genuinely large AI model, especially a genuinely large language model, on just one single individual GPU would, quite simply, take an absolutely, genuinely impractically long amount of real time — potentially many months, or, for the very largest genuinely modern models, it would frankly, genuinely not even actually be realistically possible at all, given how enormous the total required underlying computation genuinely, actually is. Distributed training genuinely, actually splits this whole enormous total workload up across many separate GPUs, and often across many entirely separate physical machines as well, all working together properly, closely, and effectively in real coordination, so the genuinely same, equivalent overall total amount of underlying work can actually get done in a dramatically shorter, genuinely far more practical, more reasonable amount of real, actual elapsed time.

Ray Train specifically, genuinely handles a lot of the real, considerable underlying complexity genuinely involved in actually doing this properly and correctly — things like properly, correctly splitting up the given training data appropriately across all of the many different available separate workers, properly, correctly coordinating and combining the many separate individual workers' own partial results back together again periodically (connecting quite directly back to the NCCL topic we already covered in the previous explanation), and properly, gracefully handling the real, genuine possibility that one particular individual

worker machine might unexpectedly, genuinely fail or crash partway through a given genuinely long-running training job, without necessarily needing to simply, wastefully restart that entire whole overall training run completely again from absolute total scratch. This kind of genuine underlying fault-tolerance and genuinely automatic recovery capability matters an enormous amount in practice, precisely because genuinely large-scale distributed training runs can often, quite realistically, genuinely take many days, or even considerably longer, to fully, completely finish, and the raw, sheer statistical likelihood of at least one single individual machine, out of potentially many hundreds or even thousands involved, actually failing at some genuine point along that whole considerable way, becomes genuinely quite meaningfully significant.

04 Distributed Inference

Distributed inference refers to the general, broader practice of actually running a given already-trained model's own predictions (generating actual real responses) across multiple separate machines or multiple separate GPUs together, rather than genuinely, exclusively relying on just one single individual GPU alone to actually handle absolutely everything, entirely by itself.

There are genuinely a couple of distinct, different reasons why distributed inference specifically becomes genuinely necessary in real practice. The first, most straightforward genuine reason is scale — connecting quite directly back to the continuous batching and GPU scheduling concepts we already covered in the previous explanation, serving a genuinely large number of simultaneous real users all at once simply, genuinely requires more total raw available GPU capacity than any one single individual GPU could ever reasonably provide entirely on its own, so incoming requests genuinely need to actually be properly spread out across many separate available GPUs together. The second, genuinely quite different reason is model size itself — connecting quite directly back to the GPU memory discussion from the earlier explanation, some of today's genuinely largest modern models are actually so enormous that they genuinely, simply don't even physically fit within one single individual GPU's own available memory capacity at all, meaning the given model itself genuinely has to actually be split up and spread properly across several separate GPUs working together, purely just to actually be able to run at all, entirely regardless of overall total user demand or overall total user traffic volume.

This second particular scenario specifically, genuinely introduces some real additional technical complexity worth being aware of — when a given single model is actually genuinely split up across multiple separate GPUs, those given separate GPUs genuinely need to actually, efficiently communicate with each other quite considerably, quite frequently, throughout the entire given process of actually generating each individual token (connecting directly back to the NCCL topic we already covered in the previous explanation), since different genuinely separate specific parts of that same overall given single model's own required calculation may genuinely, actually be physically located on entirely different separate GPUs. Getting this particular kind of cross-GPU communication genuinely fast and genuinely efficient becomes a real, quite critical, and often quite significant factor in determining the overall final real-world practical serving speed for exactly these kinds of especially large, genuinely multi-GPU models specifically.

05 Distributed Training

We already touched on distributed training conceptually, back in the Ray Train section above, but it's genuinely worth understanding the broader overall concept here somewhat more fully, since it's really the genuinely foundational underlying idea that actually makes training today's largest, most capable modern models genuinely possible at all, in the very first place.

There are genuinely a couple of distinct, different general common approaches worth understanding here. **Data parallelism** genuinely means taking one single given complete copy of the entire model, and actually placing that same complete given copy onto several genuinely separate different GPUs, with each individual separate GPU then genuinely, actually training on its own genuinely different, separate portion or "slice" of the overall total available training data, before all of those genuinely separate individual GPUs then periodically, genuinely, properly synchronize their own respective learned results back together again. **Model parallelism**, by genuine contrast, instead genuinely means splitting up the actual model itself across several genuinely separate different GPUs (connecting quite directly back to the distributed inference discussion just above), which genuinely, specifically becomes necessary whenever a given particular model itself is simply, genuinely too enormous to actually physically fit entirely within just one single individual GPU's own available memory capacity alone, regardless of the particular training data involved. Many genuinely large, real-world, modern practical training setups actually, genuinely combine both of these particular approaches properly together at once, in various different, genuinely sophisticated combined ways.

The genuinely core underlying overall challenge running throughout distributed training, regardless of exactly which particular specific approach is genuinely actually being used, remains fundamentally the same one we've already, genuinely touched on repeatedly throughout this whole entire series: efficient communication and coordination between many genuinely separate individual machines. Every single time these many separate individual GPUs genuinely need to actually properly synchronize their own respective learned results back together (which genuinely, actually happens quite considerably, quite frequently throughout any given typical overall training run), that particular given communication step itself genuinely takes real, actual time — and if that particular communication overhead genuinely isn't managed carefully, efficiently, and properly well, a team can end up in the genuinely quite frustrating, quite counterintuitive situation of actually adding considerably more total GPUs to a given overall training job, while genuinely, actually seeing surprisingly, disappointingly little corresponding real additional improvement in overall total training speed to actually show properly for it, purely because all of that given additional raw hardware ends up simply, mostly just sitting there, wastefully, uselessly waiting around on slow, inefficient communication instead.

06 Kubernetes for AI

Kubernetes is a widely-used, well-established general-purpose system for actually managing containerized applications at real, meaningful production scale — you may already genuinely recall Kubernetes coming up briefly, back in our earlier Docker security material, since Kubernetes is genuinely the tool most commonly, widely used specifically for actually running Docker containers reliably, properly, at genuine real production scale. Kubernetes for AI specifically refers to actually, genuinely using this exact same well-established, broadly general infrastructure tool, but now specifically, deliberately applied to genuinely running AI workloads in particular.

Why would an organization genuinely want to actually use Kubernetes specifically for AI work, rather than perhaps just directly, simply using something like Ray entirely on its own instead? The genuine, honest answer is that these particular tools genuinely, actually work quite well together, rather than genuinely competing directly against one another as pure, mutually-exclusive alternatives. Kubernetes already, genuinely provides a whole lot of extremely well-established, broadly general infrastructure management capability — reliably, properly scheduling work across many separate available machines, properly handling failures gracefully when they inevitably do genuinely occur, and properly managing networking and storage

reliably — capabilities that genuinely, actually matter every bit just as much for AI workloads specifically as they already genuinely do for any other kind of more regular, traditional software application generally. Rather than a given organization needing to separately, redundantly build all of that same underlying general infrastructure capability completely again from scratch, specifically and only for AI work in particular, they can instead genuinely, sensibly just run their AI-specific tools (like Ray, or the various dedicated serving engines we already covered in the previous explanation) properly right on top of Kubernetes, benefiting quite directly from all of that same already well-established, broadly general infrastructure capability that Kubernetes genuinely already, reliably provides.

The genuine catch specifically worth understanding here is that Kubernetes, in its own genuine original, more general-purpose design, genuinely wasn't originally, specifically built with AI-specific concerns like GPUs particularly, deeply in mind from its very outset — which is exactly, precisely why several of the additional specific tools we're about to cover throughout the remainder of this whole explanation genuinely, actually exist in the first place: they're specifically, deliberately designed to properly extend Kubernetes' own already broadly general capabilities, specifically adding in the particular kind of genuine, deep AI-specific awareness (like genuinely proper, correct GPU handling) that Kubernetes' own original, more general-purpose design genuinely doesn't, by itself alone, natively, fully provide right out of the box.

07 GPU Scheduling in Kubernetes

We already covered the general concept of GPU scheduling back in the previous explanation, as the broader overall process of actually deciding which specific job gets to actually use which specific available GPU. GPU scheduling in Kubernetes specifically refers to actually, genuinely making this same general underlying concept properly work correctly within the specific Kubernetes system itself, which genuinely, actually requires some real, meaningful additional specific configuration beyond what Kubernetes genuinely, natively already provides for entirely on its own, right out of the box.

Here's the genuine, specific underlying problem worth understanding clearly. Kubernetes, in its own original, more general-purpose design, already, quite naturally understands entirely regular, ordinary computing resources reasonably well — things like a given machine's own available raw CPU processing power, and its own available raw memory capacity — and it can already, quite capably, properly schedule various given ordinary workloads sensibly across a whole given cluster based specifically on those particular given resources. GPUs, however, genuinely require some real, meaningful additional specific configuration to actually be genuinely, properly understood and properly, correctly handled by Kubernetes at all — a given cluster genuinely needs to actually, specifically be told which particular specific machines genuinely, actually even have GPUs physically available at all, and Kubernetes genuinely needs to actually be able to properly, reliably ensure that a given job requesting a genuine GPU actually, genuinely gets properly scheduled specifically onto one of those particular given GPU-equipped machines, rather than potentially mistakenly ending up scheduled instead onto some other given regular, ordinary machine that genuinely, actually has no GPU physically present at all.

Beyond simply, basically just properly recognizing that GPUs genuinely, physically exist at all in the first place, more genuinely, properly sophisticated GPU scheduling also generally needs to properly, sensibly handle a number of additional, real, more nuanced practical considerations — like properly, correctly handling situations where a given individual job genuinely only actually needs to use just part of one single given available GPU, rather than an entire whole one (potentially allowing several genuinely separate different smaller given jobs to actually properly, sensibly share just one single given available GPU together), and properly, correctly handling something genuinely called "gang scheduling" — meaning making

genuinely sure that all of the various different multiple GPUs genuinely required together, all at once, for one single given larger distributed training job all genuinely, actually become properly available together, all simultaneously, rather than the given job potentially, mistakenly starting with only some, but genuinely, actually not all, of its own genuinely required total GPUs actually yet properly available at that given particular moment.

08 NVIDIA GPU Operator

The NVIDIA GPU Operator is a specific tool that specifically, genuinely automates the process of actually, properly getting a whole given Kubernetes cluster genuinely properly set up to actually, correctly work well with NVIDIA GPUs — connecting quite directly back to the whole NVIDIA driver stack, CUDA toolkit, and cuDNN topics we already covered together at real length in the previous explanation.

Here's the genuine, specific underlying problem it actually, specifically solves. Recall from the previous explanation that actually getting a GPU genuinely properly, correctly working requires several genuinely separate, distinct software layers all genuinely, properly installed correctly and genuinely properly matching each other's specific respective versions — the actual underlying driver itself, the CUDA toolkit, and several other genuinely additional, related supporting pieces as well. Now genuinely, actually imagine having to properly do all of that same considerable underlying setup work correctly, by hand, entirely manually, across potentially many hundreds of genuinely separate individual machines that together make up one single given large Kubernetes cluster — this would genuinely, honestly be an absolutely, genuinely enormous, extremely tedious, and quite genuinely error-prone undertaking to actually properly do correctly, entirely by hand.

The GPU Operator specifically, genuinely automates this entire whole considerable process — automatically, properly detecting which particular given machines within a whole given cluster genuinely actually have GPUs physically present at all, automatically properly installing the genuinely correct, properly matching driver and genuinely correct, properly matching CUDA versions onto each of those particular given machines, and generally properly, reliably keeping this whole entire considerable underlying software stack genuinely properly, correctly maintained and genuinely properly kept up to date over real, ongoing continuing time. This particular kind of genuine automation matters an enormous amount in practice, precisely because it genuinely, considerably reduces both the real, considerable initial setup effort genuinely required, and, importantly, the genuine, real ongoing maintenance burden genuinely required, specifically for actually, properly running GPU-equipped machines reliably, properly, at real, genuine, meaningful production scale.

09 KServe

KServe is a specialized tool that's specifically, genuinely built directly on top of Kubernetes, specifically and deliberately focused on actually, properly serving AI models within that same given Kubernetes environment — it started life originally as a genuinely component part of a larger, broader project called KubeFlow (which we'll actually properly cover just next, right below), before it later actually, genuinely became independently spun off, and is now maintained separately, all on its own.

The genuine, core underlying idea behind KServe is to provide a genuinely standardized, consistent way of actually, properly deploying and properly, reliably managing model-serving endpoints properly within Kubernetes, regardless of exactly which particular given underlying serving engine (like vLLM, TensorRT-LLM, or any of the other several specific tools we already covered together in the previous explanation) actually, genuinely happens to actually be doing the real, raw underlying work of actually running that

particular given model. It provides a well-defined, genuinely standardized, common way of properly, cleanly describing "here's my given model, here's specifically where it's actually properly stored, and here's genuinely how I'd generally like it to actually properly, reliably scale" — and KServe itself then genuinely, properly handles a lot of the real, considerable underlying complexity genuinely involved in actually, properly making that same given description happen correctly, reliably, in genuine practice.

A genuinely particularly valuable specific feature that KServe specifically, genuinely provides is "scale-to-zero" capability — meaning that for a given model endpoint that genuinely, actually isn't currently, actively being used by anyone at any genuinely given particular moment, KServe can genuinely, actually reduce its own given running resources properly all the way down to literally, genuinely zero, and then automatically, properly bring it back genuinely, properly up again as soon as a genuinely new incoming request actually, genuinely arrives. This particular capability genuinely, considerably matters for real cost efficiency, connecting quite directly back to the cost optimization concepts we already covered at real length back in the earlier Week 9 material — there's genuinely, honestly no real, meaningful practical reason to keep an expensive, given GPU sitting there, wastefully, uselessly running and fully, actively powered on, purely idle, for some particular given model that genuinely, actually isn't currently, actually being properly used by anyone at all, at that given particular specific moment in real time.

10 Kubeflow

Kubeflow is a genuinely more complete, considerably broader Kubernetes-native machine learning platform, and it's genuinely worth understanding as being meaningfully broader in its own overall scope compared to some of the more narrowly-focused, individual specific tools we've already covered so far throughout this whole explanation. Rather than genuinely, exclusively focusing on just any one single specific particular piece of the overall broader machine learning lifecycle alone, Kubeflow instead genuinely, actually bundles together several genuinely distinct, separate components, specifically designed to properly, comprehensively cover the entire whole overall machine learning lifecycle together, all as one single, unified, cohesive combined package.

Kubeflow's own particular included components genuinely cover a lot of considerable, real ground together — a dedicated Pipelines component genuinely helps properly orchestrate the various multi-step workflows genuinely involved in actually, properly preparing given training data, actually properly training a given model, and then actually, properly evaluating it afterward; a dedicated Training Operator component genuinely helps actually, properly run genuinely distributed training jobs properly (connecting quite directly back to the distributed training concepts we already covered above, earlier in this same explanation); and KServe itself, which we already just properly covered right above, actually genuinely handles the final resulting model-serving piece of this same overall combined package. Together, all of these various genuinely separate individual components genuinely aim to provide one single, complete, unified, end-to-end platform specifically covering the entire overall machine learning lifecycle, all properly running natively right on top of Kubernetes itself.

The genuine tradeoff worth understanding here, compared to something like Ray (which we already, actually covered near the very beginning of this whole explanation), is that Kubeflow's own considerably broader, more comprehensive overall scope genuinely, correspondingly comes together with real, meaningful additional operational complexity — actually properly running the entire whole full Kubeflow stack genuinely, actually requires real, dedicated, considerable ongoing platform engineering effort and real, genuine expertise to properly maintain reliably, over real, considerable ongoing time. Because of this, Kubeflow tends to genuinely, actually be a particularly strong, well-suited fit specifically for genuinely larger organizations

that have real, dedicated platform teams genuinely available, and that genuinely, actually want one single, complete, unified, comprehensive platform properly covering their entire whole overall machine learning workflow together, rather than genuinely needing to separately, individually assemble and properly maintain several genuinely different, separate individual specialized tools entirely on their own, each one covering just its own particular, individual specific separate piece.

11 Autoscaling AI Workloads

Autoscaling refers to the genuinely important, general capability of a given system to actually, automatically adjust how much total underlying computing capacity it's genuinely, currently actually using, in direct response to genuinely, actually changing real-time demand — automatically, properly scaling up when genuine real traffic or demand actually genuinely increases, and correspondingly, properly automatically scaling back down again whenever that same given real demand genuinely, actually decreases again afterward.

This particular capability genuinely, considerably matters an enormous amount specifically for AI workloads in real practice, precisely because real-world genuine demand for a given AI application often, genuinely varies quite considerably over real, ongoing time — a genuinely popular given customer-facing application might genuinely, actually experience considerably heavier real traffic during certain particular given times of day, and considerably lighter real traffic during others. Without genuinely, properly effective autoscaling actually properly in place, a given team genuinely faces an uncomfortable, real practical tradeoff: either genuinely, actually provisioning enough total available GPU capacity to properly, comfortably handle their given absolute peak level of real demand at all times (which genuinely, actually means a lot of that same expensive, considerable GPU capacity sits there, wastefully, uselessly idle, most of the actual real time), or instead genuinely, actually provisioning for only more typical, average real demand levels (which then genuinely, actually risks real, meaningful poor performance or genuine, real service outages specifically whenever real demand genuinely, actually happens to spike considerably higher than that given average level).

Autoscaling specifically for AI workloads genuinely, actually introduces some real, particular additional complexity worth being genuinely aware of, well beyond regular, more traditional software autoscaling generally. GPUs genuinely, actually take real, meaningfully longer to actually properly "warm up" and genuinely become properly ready compared to a considerably more lightweight, regular traditional web server generally would — a given large model genuinely needs real, actual meaningful time to actually properly load into a given GPU's own available memory before it can genuinely, actually begin properly serving any real requests at all, meaning autoscaling genuinely decisions specifically for AI workloads genuinely, actually need to properly, sensibly account for this same real, meaningful additional given startup delay, connecting quite directly back to that same scale-to-zero capability we already, specifically discussed together, back in the KServe section above — genuinely, properly balancing real cost savings achieved specifically through actually properly scaling all the way down, against the real, corresponding tradeoff of genuinely somewhat slower given response times specifically for that same very first given request that genuinely, actually arrives right after any given period of genuine, real actual full idle time.

12 AI Infrastructure Monitoring

We already, actually covered AI observability at real, considerable length together, back in the earlier Week 9 material — but that particular earlier discussion genuinely, actually focused specifically and primarily on monitoring a given AI application's own overall behavior and output quality specifically. AI infrastructure monitoring specifically refers instead to monitoring the actual underlying hardware and infrastructure itself — genuinely, actually watching how well the given actual GPUs, the given actual overall cluster, and the given actual overall serving systems we've properly covered throughout this whole entire Week 12 material are all actually, genuinely performing, at that same particular given underlying infrastructure level specifically.

This particular kind of infrastructure-level monitoring generally, genuinely tracks several distinct, genuinely important specific things together. **GPU utilization** genuinely measures how genuinely, actually busy a given available GPU actually, genuinely is at any given particular moment — a given GPU that's genuinely, actually sitting there mostly idle represents real, genuine wasted money, connecting quite directly back to the whole cost allocation and cost optimization concepts we already, properly covered together back in the earlier Week 9 and Week 10 material. **Memory utilization** genuinely tracks how much of a given available GPU's own particular memory capacity is genuinely, actually currently actually being used, connecting quite directly back to the KV cache and general GPU memory discussions we've already, properly covered together throughout this whole entire Week 12 material — genuinely, actually running consistently too close to that same given available memory limit genuinely, actually risks real, genuine request failures. **Cluster health** genuinely tracks whether the various individual machines making up a given overall broader cluster are all genuinely, actually properly functioning correctly, and whether the given underlying GPU drivers and given underlying CUDA software stack we already, properly covered together earlier remain genuinely, actually properly, correctly functioning as they genuinely, actually should be. And **queue depth and scheduling delays** genuinely track how genuinely, actually long given individual jobs are genuinely, actually having to properly wait around before they can genuinely, actually finally actually begin properly running at all, connecting quite directly back to the whole GPU scheduling concepts we've already, properly covered together throughout this whole entire explanation.

The genuinely important, broader overall reason this particular kind of infrastructure-level monitoring genuinely, actually matters so considerably, so much, connects quite directly back to a genuinely, actually recurring broader theme that's genuinely run consistently throughout this whole entire series: you genuinely can't effectively, properly optimize or genuinely, properly meaningfully improve what you genuinely, actually can't properly, clearly see and properly measure in the very first place. A given team that genuinely, actually lacks proper, genuine visibility into how well their own given underlying GPU infrastructure is genuinely, actually actually performing has genuinely, actually no real, reliable way of genuinely knowing whether they're actually, genuinely getting proper, full, appropriate value for the genuinely considerable money they're actually, genuinely spending on that same given expensive infrastructure, and genuinely, actually has no reliable, proper early warning at all whenever some given real, genuine underlying problem first genuinely, actually begins to properly emerge — which brings this whole entire twelve-week series full circle, back around to that same genuinely, consistently recurring core theme we've genuinely, properly emphasized quite repeatedly throughout this entire whole broader course: building genuinely good AI systems was genuinely, actually never truly just about getting them to genuinely work well just the one single time — it's genuinely about the entire whole complete, ongoing discipline of properly, reliably keeping them genuinely working well, at every single given layer of the whole entire overall stack, from the very highest-level given prompt, all the way properly down to the actual given raw physical silicon chip itself, genuinely, reliably, over real, considerable, extended given time.